

Assignment: Teacher and Student Registration Using Express.js and MongoDB

Problem Statement

Create an **Express.js** application that allows **teachers and students to register** and store their details in a MongoDB database using **Mongoose**.

Students must create **separate schemas, models, and routers** for teachers and students.

Requirements

1. Create the following folder structure:

```
project/
├── server.js
├── schema/
│   ├── teacherSchema.js
│   └── studentSchema.js
├── model/
│   ├── teacherModel.js
│   └── studentModel.js
└── router/
    ├── teacherRouter.js
    └── studentRouter.js
```

2. Connect the Express.js application with MongoDB using Mongoose.
3. Create a **Teacher Schema** with suitable fields such as:
 - Name
 - Email
 - Password
 - Subject
4. Create a **Student Schema** with suitable fields such as:
 - Name
 - Email

- Password
- Course
- Age

5. Create two registration endpoints:

POST /teacher/register

POST /student/register

6. Validate the registration data before storing it.
7. Hash the password using **bcrypt** before storing it in MongoDB.
8. Store teacher and student data in their respective MongoDB collections.
9. Return an appropriate success or error message.

Expected Flow

Teacher

POST /teacher/register



Teacher Schema Validation



Hash Password



Teacher MongoDB Collection

Student

POST /student/register



Student Schema Validation



Hash Password



Student MongoDB Collection

Submission Requirements

Students must submit:

- Complete source code
- Proper folder structure
- Screenshot of successful MongoDB connection
- Screenshot of teacher registration in Postman
- Screenshot of student registration in Postman
- Screenshot showing teacher data in MongoDB
- Screenshot showing student data in MongoDB
- Screenshot showing passwords stored in **hashed form**

Rubric – 10 Marks

Criteria	Description	Marks
MongoDB Connection	Successfully connects Express.js with MongoDB using Mongoose	1
Folder Structure	Creates separate schema, model and router files for teacher and student	2
Teacher Registration	Correctly implements teacher schema and <code>POST /teacher/register</code>	2
Student Registration	Correctly implements student schema and <code>POST /student/register</code>	2
Validation	Properly validates teacher and student registration data	1
Password Hashing	Hashes passwords using bcrypt before storing	1
Code Quality & Screenshots	Clean code and required screenshots are provided	1
Total		10